

POSTGRESQL · JOIN · SUBQUERY · WINDOW FUNCTION

PostgreSQL 종합실습 2

SQL Query 및 실행 결과

학사 시스템, 캠퍼스 스토어, 조직도 데이터를 활용한 JOIN·집계·서브쿼리·CTE·Window Function 실습

소속

광주 3반

이름

신형섭

작성일

2026-07-23

DBMS

PostgreSQL 17

도구

VS Code · DBeaver

문항

1번 ~ 25번

JOIN 기본·확장

INNER · LEFT · FULL · SELF · CROSS JOIN

서브쿼리·집합

Scalar · Correlated · EXISTS · UNION

심화 SQL

ROLLUP · Recursive CTE · Window Function

제출물 구성 — 각 문항의 최종 SQL과 PostgreSQL 실행 결과를 한 화면에 캡처하여 문항 순서대로 정리했다.

LAB OVERVIEW

실습 환경 및 학습 목표

02

공통 데이터 영역

- student · enroll — 학생, 학과, GPA, 과목 및 성적
- customers · orders — 고객별 주문 및 주문 금액
- emp — CEO, 매니저, 개발자의 자기참조 조직도
- course_owner — 과목과 책임 매니저의 실습용 매핑

핵심 학습 목표

- JOIN 방향에 따른 포함 범위와 NULL 발생 원리 이해
- GROUP BY와 집계 함수를 이용한 리포트 작성
- 스칼라·상관 서브쿼리와 EXISTS 활용
- ROLLUP, 재귀 CTE, 순위·누적 Window Function 적용

JOIN
1~10매핑·CROSS
11~13서브쿼리
14~20ROLLUP·CTE
21~22Window
23~25

요구사항 반영

문항별 JOIN 종류, 집계 기준, 서브쿼리 위치 및 Window Frame을 SQL에 반영했다.

컬럼 최소화

결과 해석에 필요한 식별자와 요구 컬럼만 선택해 불필요한 출력을 줄였다.

결과 검증

각 문항의 실행 화면에 최종 SQL과 결과 컬럼 및 샘플 행을 함께 기록했다.

출력 기준

일반 조회는 5건, 상위 고객은 10건, 전체 조합은 100건 등 문항별 범위를 적용했다.

실습 결과 요약

단순 조회에서 출발해 계층 탐색, 그룹 소개·총계, 순위, 이전 행 비교, 누적합과 이동평균까지 PostgreSQL의 주요 분석 패턴을 단계적으로 구현했다.

수강 데이터 연결 범위

01 수강 기록이 존재하는 학생 조회

```

-- 1번
-- 학생과 수강을 INNER JOIN하여 수강 기록이 존재하는 학생의 과목과 성적을
-- 조회한다.
-- */
SELECT s.student_id, s.name, e.course, e.grade
FROM student s
INNER JOIN enroll e on s.student_id = e.student_id
ORDER BY s.student_id
LIMIT 5;

```

	student_id	AZ name	AZ course	AZ grade
1	1	Student_1	Course_3	C
2	1	Student_1	Course_4	D
3	1	Student_1	Course_5	A
4	2	Student_2	Course_4	D
5	2	Student_2	Course_5	A

INNER JOIN — student와 enroll 양쪽에 모두 존재하는 학생의 과목과 성적을 조회했다.

02 모든 학생을 기준으로 수강 정보 연결

```

-- 2번
-- 모든 학생을 기준으로 수강 정보를 연결한다.
-- - 수강한 과목이 없는 학생도 결과에 포함한다.
-- - 과목이 없는 경우 과목 관련 정보가 NULL로 표시되도록 한다.
-- */
SELECT s.student_id, s.name, e.course, e.grade
FROM student s
LEFT JOIN enroll e on s.student_id = e.student_id
ORDER BY s.student_id
LIMIT 5;

```

	student_id	AZ name	AZ course	AZ grade
1	1	Student_1	Course_3	C
2	1	Student_1	Course_4	D
3	1	Student_1	Course_5	A
4	2	Student_2	Course_4	D
5	2	Student_2	Course_5	A

LEFT JOIN — 수강 기록이 없는 학생도 유지하고 과목 정보는 NULL로 표현했다.

RIGHT-SIDE BASE · FULL OUTER JOIN

고아 수강 및 양쪽 데이터 조회

04

03 수강 정보를 기준으로 학생 연결

```

/*
3번
수강 정보를 기준으로 조회한다.
- 대응하는 학생이 없는 고아 수강 데이터도 결과에 포함한다.
- 학생이 없으면 학생 정보가 NULL로 표시되도록 한다.
*/
SELECT
  e.student_id AS enroll_student_id,
  s.student_id AS student_id,
  s.name,
  e.course,
  e.grade
FROM enroll e
LEFT JOIN student s on e.student_id = s.student_id
ORDER BY e.course, e.grade
LIMIT 5;

```

enroll(+)	1	X
enroll(+)	1	X
123 enroll_student_id	123 student_id	AZ name
1001	[NULL]	[NULL]
37	37	Student_37
17	17	Student_17
18	18	Student_18
19	19	Student_19

enroll 기준 LEFT JOIN — 학생 테이블에 없는 고아 수강의 원래 ID와 NULL 학생 정보를 함께 확인했다.

04 학생과 수강 양쪽의 전체 데이터 조회

```

/*
4번
학생 데이터와 수강 데이터를 양쪽 모두 포함하여 조회한다.
- 학생만 존재하는 데이터도 포함한다.
- 수강 정보만 존재하는 데이터도 포함한다.
*/
SELECT
  e.student_id AS enroll_student_id,
  s.student_id AS student_id,
  s.name,
  e.course,
  e.grade
FROM student s
FULL OUTER JOIN enroll e on e.student_id = s.student_id
ORDER BY COALESCE(s.student_id, e.student_id), e.course
LIMIT 5;

```

enroll(+)	1	X
enroll(+)	1	X
123 enroll_student_id	123 student_id	AZ name
1	1	Student_1
1	1	Student_1
1	1	Student_1
2	2	Student_2
2	2	Student_2

FULL OUTER JOIN — 학생만 있는 행과 수강만 있는 행을 모두 포함했다.

LEFT ANTI JOIN · DISTINCT

05

수강 여부에 따른 학생 분류

05 한 번도 수강하지 않은 학생

```

-- 5번
-- 한 번도 수강하지 않은 학생 목록을 조회한다.
-- */
SELECT s.student_id, s.name
FROM student s
LEFT JOIN enroll e ON e.student_id = s.student_id
WHERE e.student_id IS NULL
ORDER BY s.student_id
LIMIT 5;

```

student 1 X

SELECT s.student_id, s.name FROM student s LEFT JOIN enroll e ON e.student_id = s.student_id

student_id	name
10	Student_10
20	Student_20
30	Student_30
40	Student_40
50	Student_50

LEFT JOIN + IS NULL — enroll에 대응 행이 없는 학생만 선택했다.

06 한 과목 이상 수강한 학생

```

-- 6번
-- 한 과목 이상 수강한 학생 목록을 조회한다.
-- - 동일 학생이 여러 수강 기록을 가지더라도 학생은 중복되지 않도록 한다.
-- */
SELECT DISTINCT s.student_id, s.name
FROM student s
INNER JOIN enroll e ON e.student_id = s.student_id
ORDER BY s.student_id
LIMIT 5;

```

student 1 X

SELECT s.student_id, s.name FROM student s INNER JOIN enroll e ON e.student_id = s.student_id

student_id	name
10	Student_10
20	Student_20
30	Student_30
40	Student_40
50	Student_50

INNER JOIN + DISTINCT — 수강 과목 수와 관계없이 학생 한 명을 한 번만 출력했다.

GROUP BY · COUNT · SUM

06

고객별 주문 현황과 상위 고객

07 고객별 주문 건수와 총 주문 금액

```

/*
7번
고객별 주문 건수와 총 주문 금액을 조회한다.
*/
SELECT cus.customer_id, cus.customer_name, COUNT(o.order_id) AS order_count, SUM(o.
FROM customers cus
INNER JOIN orders o on cus.customer_id = o.customer_id
GROUP BY cus.customer_id, cus.customer_name
ORDER BY cus.customer_id
LIMIT 5;

```

student 1 X

```

SELECT s.student_id, s.name FROM student s LEFT JOIN enroll e ON e.st
Enter a SQL expression to fi

```

	student_id	AZ name
1	10	Student_10
2	20	Student_20
3	30	Student_30
4	40	Student_40
5	50	Student_50

GROUP BY — 고객 식별 정보별로 주문 건수와 금액 합계를 계산했다.

08 총 주문 금액 상위 10명

```

/*
8번
총 주문 금액이 높은 고객 상위 10명과 해당 금액을 조회한다.
*/
SELECT cus.customer_id, cus.customer_name, SUM(o.amount) AS total_amount
FROM customers cus
INNER JOIN orders o on cus.customer_id = o.customer_id
GROUP BY cus.customer_id, cus.customer_name
ORDER BY total_amount DESC, cus.customer_id
LIMIT 10;

```

customers 1 X

```

SELECT cus.customer_id, cus.customer_name, SUM(o.amount) AS total_
Enter a SQL expression to

```

	customer_id	AZ customer_name	total_amount
1	154	Customer_154	19,467.18
2	153	Customer_153	19,389.12
3	152	Customer_152	19,311.06
4	151	Customer_151	19,233
5	150	Customer_150	19,154.94
6	149	Customer_149	19,076.88
7	148	Customer_148	18,998.82
8	147	Customer_147	18,920.76
9	146	Customer_146	18,842.7
10	145	Customer_145	18,764.64

ORDER BY total_amount DESC — 금액 내림차순과 고객 ID 2차 정렬로 상위 10명을 추출했다.

SELF JOIN · LEFT JOIN AGGREGATION

07

조직도와 수강 분포

09 직원과 매니저 이름

```

-- /*
-- 9번
-- -----
-- 모든 직원과 각 직원의 매니저 이름을 조회한다.
-- - CEO처럼 매니저가 없는 직원도 결과에 포함한다.
-- */
SELECT e.emp_id, e.name AS employee_name, m.name AS manager_name
FROM emp e
LEFT JOIN emp m on e.manager_id = m.emp_id
ORDER BY e.emp_id
LIMIT 5;

```

emp 1	emp_id	employee_name	manager_name
1	1	CEO	[NULL]
2	2	Mgr_2	CEO
3	3	Mgr_3	CEO
4	4	Mgr_4	CEO
5	5	Mgr_5	CEO

SELF LEFT JOIN — emp를 직원과 매니저 역할로 나누고 CEO의 매니저는 NULL로 유지했다.

10 과목별 수강 분포

```

-- /*
-- 10번
-- -----
-- 모든 학생을 기준으로 과목별 수강 분포를 확인한다.
-- - LEFT JOIN과 집계를 사용한다.
-- - 수강 기록이 없는 학생도 결과 분석에 포함한다.
-- */
SELECT e.course, COUNT(s.student_id) AS student_count
FROM student s
LEFT JOIN enroll e on s.student_id = e.student_id
GROUP BY e.course
ORDER BY e.course NULLS FIRST
LIMIT 5;

```

enroll 1	course	student_count
	[NULL]	100
	Course_1	144
	Course_10	96
	Course_11	144
	Course_12	48

LEFT JOIN + GROUP BY — 미수강 학생을 NULL 과목 그룹에 포함해 분포를 집계했다.

CONDITIONAL JOIN · MAPPING TABLE

08

DB 미수강자와 과목 책임자

11 DB 과목을 듣지 않은 학생

11번

DB 과목을 듣지 않은 모든 학생을 나열한다.

```

/*
SELECT s.student_id, s.name
FROM student s
LEFT JOIN enroll e
  ON s.student_id = e.student_id
  AND e.course = 'DB'
WHERE e.student_id IS NULL
ORDER BY s.student_id
LIMIT 5;
*/

```

student 1 X

SELECT s.student_id, s.name FROM student s LEFT JOIN enroll e ON s.student_id = e.student_id AND e.course = 'DB' WHERE e.student_id IS NULL ORDER BY s.student_id LIMIT 5;

	123 student_id	AZ name
1	1	Student_1
2	2	Student_2
3	3	Student_3
4	4	Student_4
5	5	Student_5

ON 조건 + IS NULL — DB 수강 행만 연결한 뒤 연결되지 않은 학생을 선택했다.

12 과목별 수강 인원과 책임 매니저

12번

과목별 매니저가 운영 책임을 가진다고 가정하여 리포트를 작성한다.

요구사항:

- emp 테이블에서 이름이 Mgr_로 시작하는 매니저를 사용한다.
- 매니저와 과목을 임의로 매핑하는 테이블을 만든다.
- 매핑 테이블 이름은 course_owner로 한다.
- course_owner의 주요 컬럼은 다음과 같다.
 - course
 - manager_id
- 과목별 수강 인원과 책임 매니저 이름을 조회하는 리포트를 작성한다.

```

/*
CREATE TABLE course_owner (
  course VARCHAR(50),
  manager_id INT
);

INSERT INTO course_owner (course, manager_id)

SELECT co.course,
  COUNT(DISTINCT e.student_id) AS student_count,
  e.name AS manager_name
FROM course_owner co
LEFT JOIN enroll e ON co.course = e.course
INNER JOIN emp m ON co.manager_id = m.emp_id
GROUP BY co.course, m.name
ORDER BY co.course
LIMIT 5;
*/

```

course_owner(+) 1 X

SELECT co.course, COUNT(DISTINCT e.student_id) AS student_count, e.name AS manager_name FROM course_owner co LEFT JOIN enroll e ON co.course = e.course INNER JOIN emp m ON co.manager_id = m.emp_id GROUP BY co.course, m.name ORDER BY co.course LIMIT 5;

	AZ course	123 student_count	AZ manager_name
1	AI	1	Mgr_3
2	Course_1	144	Mgr_5
3	Course_10	96	Mgr_4
4	Course_11	144	Mgr_5
5	Course_12	48	Mgr_6

course_owner — 과목과 매니저를 매핑하고 수강 인원과 책임자 이름을 집계했다.

CARTESIAN PRODUCT · SELECT SUBQUERY

09

추천 후보와 소속 학과 조회

13 학생과 과목의 전체 조합

13번

학생과 과목의 전체 조합을 만들어 학생별 과목 추천 후보를 생성한다.

- 학생과 과목의 모든 조합을 만든다.
- 샘플 결과는 100건만 조회한다.

```

/*
SELECT
  s.student_id,
  s.name,
  c.course
FROM student s
CROSS JOIN (
  SELECT DISTINCT course
  FROM enroll
) c
ORDER BY s.student_id, c.course
LIMIT 100;

```

student_id	name	course
1	Student_1	AI
2	Student_1	Course_1
3	Student_1	Course_10
4	Student_1	Course_11
5	Student_1	Course_12
6	Student_1	Course_13
7	Student_1	Course_14
8	Student_1	Course_15
9	Student_1	Course_16
10	Student_1	Course_17
11	Student_1	Course_18
12	Student_1	Course_19
13	Student_1	Course_2
14	Student_1	Course_20
15	Student_1	Course_3
16	Student_1	Course_4
17	Student_1	Course_5
18	Student_1	Course_6
19	Student_1	Course_7
20	Student_1	Course_8
21	Student_1	Course_9
22	Student_1	DB
23	Student_1	ML
24	Student_2	AI
25	Student_2	Course_1

CROSS JOIN — 학생 × 교유 과목의 전체 조합에서 100건을 조회했다.

14 SELECT 절의 스칼라 서브쿼리

14번

스칼라 서브쿼리, 즉 SELECT 절의 서브쿼리를 사용하여 학생과 소속 학과를 함께 조회한다.

```

/*
SELECT
  s.student_id,
  s.name,
  (
    SELECT s2.major
    FROM student s2
    WHERE s2.student_id = s.student_id
  ) AS major
FROM student s
ORDER BY s.student_id
LIMIT 5;

```

student_id	name	major
1	Student_1	EE
2	Student_2	ME
3	Student_3	CE
4	Student_4	BIO
5	Student_5	CS

Scalar Subquery — 학생별 단일 학과 값을 SELECT 절에서 조회했다.

AVERAGE COMPARISON

전체 및 학과 평균 GPA 비교

10

15 전체 평균 GPA보다 높은 학생

```

-- 15번
-- 전체 학생의 평균 GPA보다 GPA가 높은 학생을 조회한다.
-- WHERE 절의 서브쿼리를 사용한다.
*/
SELECT s.student_id, s.name, s.gpa
FROM student s
WHERE s.gpa > (
    SELECT AVG(s2.gpa)
    FROM student s2
)
ORDER BY s.student_id
LIMIT 5;
/*

```

student 1 X

SELECT s.student_id, s.name, s.gpa FROM student s WHERE | Enter a SQL expression to filter result

	123 student_id	AZ name	123 gpa
1	15	Student_15	3.5
2	16	Student_16	3.6
3	17	Student_17	3.7
4	18	Student_18	3.8
5	19	Student_19	3.9

WHERE Subquery — 전체 평균을 한 번 계산해 학생 GPA 필터에 적용했다.

16 자신의 학과 평균보다 높은 학생

```

-- 16번
-- 각 학생의 GPA를 해당 학생이 속한 학과의 평균 GPA와 비교한다.
-- 자신의 학과 평균 GPA보다 높은 학생을 조회한다.
-- 상관 서브쿼리인 Correlated subquery를 사용한다.
*/
SELECT s.student_id, s.name, s.major, s.gpa
FROM student s
WHERE s.gpa > (
    SELECT AVG(s2.gpa)
    FROM student s2
    WHERE s2.major = s.major
)
ORDER BY s.student_id
LIMIT 5;
/*

```

student 1 X

SELECT s.student_id, s.name, s.major, s.gpa FROM student s | Enter a SQL expression

	123 student_id	AZ name	AZ major	123 gpa
1	15	Student_15	CS	3.5
2	16	Student_16	EE	3.6
3	17	Student_17	ME	3.7
4	18	Student_18	CE	3.8
5	19	Student_19	BIO	3.9

Correlated Subquery — 외부 학생의 major를 내부 평균 계산과 연결했다.

SEMI JOIN · ANTI JOIN

11

수강 기록의 존재 여부

17 수강 기록이 존재하는 학생

```

/*
17번
수강 기록이 존재하는 학생만 조회한다.
*/
SELECT s.student_id, s.name
FROM student s
WHERE EXISTS (
    SELECT 1
    FROM enroll e
    WHERE e.student_id = s.student_id
)
ORDER BY s.student_id
LIMIT 5;

```

	student_id	name
1	1	Student_1
2	2	Student_2
3	3	Student_3
4	4	Student_4
5	5	Student_5

EXISTS — 수강 행의 실제 값 대신 존재 여부만 검사해 학생 중복을 방지했다.

18 한 번도 수강하지 않은 학생

```

/*
18번
한 번도 수강하지 않은 학생을 조회한다.
*/
SELECT s.student_id, s.name
FROM student s
WHERE NOT EXISTS (
    SELECT 1
    FROM enroll e
    WHERE e.student_id = s.student_id
)
ORDER BY s.student_id
LIMIT 5;

```

	student_id	name
1	10	Student_10
2	20	Student_20
3	30	Student_30
4	40	Student_40
5	50	Student_50

NOT EXISTS — 대응 수강 행이 전혀 없는 학생만 조회했다.

COMPARISON · SET OPERATOR

학과 비교와 조건 결합

12

19 HR 학과 학생

19번

HR 학과 학생 일부와 다른 학과 학생 일부를 비교하는 데모 쿼리를 작성한다.

```

--
SELECT s.student_id, s.name, s.major, s.gpa
FROM student s
WHERE s.major = 'HR'
ORDER BY s.student_id
LIMIT 5;

--
SELECT s.student_id, s.name, s.major, s.gpa
FROM student s
WHERE s.major <> 'HR'
ORDER BY s.student_id
LIMIT 5;

```

student 1 X

	123 student_id	AZ name	AZ major	123 gpa
1	981	Student_981	HR	4.1
2	982	Student_982	HR	4.2
3	983	Student_983	HR	4.3
4	984	Student_984	HR	4.4
5	985	Student_985	HR	4.5

HR 학과 학생 표본을 조회했다.

19 비-HR 학과 학생

19번

HR 학과 학생 일부와 다른 학과 학생 일부를 비교하는 데모 쿼리를 작성한다.

```

--
SELECT s.student_id, s.name, s.major, s.gpa
FROM student s
WHERE s.major = 'HR'
ORDER BY s.student_id
LIMIT 5;

--
SELECT s.student_id, s.name, s.major, s.gpa
FROM student s
WHERE s.major <> 'HR'
ORDER BY s.student_id
LIMIT 5;

```

student 1 X

	123 student_id	AZ name	AZ major	123 gpa
1	1	Student_1	EE	2.1
2	2	Student_2	ME	2.2
3	3	Student_3	CE	2.3
4	4	Student_4	BIO	2.4
5	5	Student_5	CS	2.5

동일 컬럼으로 다른 학과 표본을 조회해 비교했다.

20 CS 학과 또는 DB 수강 학생

20번

다음 조건 중 하나를 만족하는 학생 목록을 조회한다.

- CS 학과 학생
- DB 과목을 수강한 학생

```

--
SELECT s.student_id, s.name, s.major
FROM student s
WHERE s.major = 'CS'

UNION

--
SELECT s.student_id, s.name, s.major
FROM student s
INNER JOIN enroll e ON e.student_id = s.student_id
WHERE e.course = 'DB'

--
ORDER BY student_id
LIMIT 5;

```

results 1 X

	123 student_id	AZ name	AZ major
5	Student_5	CS	
10	Student_10	CS	
15	Student_15	CS	
19	Student_19	BIO	
20	Student_20	CS	

UNION — CS 학과 학생 집합과 DB 과목 수강 학생 집합을 중복 없이 결합했다.

ROLLUP · GROUPING

학과별 GPA 구간·소계·총계

13

21번 GPA를 세 구간으로 분류하고 학과 소계와 전체 총계를 한 번에 계산

25 rows

21번

student 테이블에는 학과 major와 GPA가 있다.

학과별 GPA 구간별 인원을 집계하고, 소계와 총계를 한 번의 쿼리로 출력한다.

요구사항:

- GPA 구간을 나타내는 파생 컬럼 gpa_tier를 추가한다.
- GPA 구간은 다음과 같이 분류한다.
 - 3.0 미만
 - 3.0 이상 3.5 이하
 - 3.5 초과
- GROUP BY ROLLUP(major, gpa_tier)을 이용해 다음 결과를 함께 조회한다.
 - 학과별 GPA 구간 집계
 - 학과별 소계
 - 전체 총계
- GROUPING(major) 함수를 사용한다.
- 소계 또는 총계 행의 학과 표시에는 전체 라벨을 사용한다.
- 결과는 major, gpa_tier 순으로 정렬한다.
- 소계 행은 각 학과 결과의 하단에 표시한다.

*/

```
WITH student_tier AS (
  SELECT
    major,
    CASE
      WHEN gpa < 3.0 THEN '3.0 미만'
      WHEN gpa <= 3.5 THEN '3.0 이상 3.5 이하'
      ELSE '3.5 초과'
    END AS gpa_tier
  FROM student
)
SELECT
  CASE
    WHEN GROUPING(st.major) = 1
    OR GROUPING(st.gpa_tier) = 1
    THEN '전체'
    ELSE st.major
  END AS major,
  CASE
    WHEN GROUPING(st.major) = 1 THEN '전체 총계'
    WHEN GROUPING(st.gpa_tier) = 1 THEN '학과 소계'
    ELSE st.gpa_tier
  END AS gpa_tier,
  COUNT(*) AS student_count
FROM student_tier st
GROUP BY ROLLUP(st.major, st.gpa_tier)
ORDER BY
  GROUPING(st.major),
  st.major,
  GROUPING(st.gpa_tier),
  st.gpa_tier;
```

Results 1 X

Enter a SQL expression to filter results

	AZ major	AZ gpa_tier	123 student_count
1	BIO	3.0 미만	66
2	BIO	3.0 이상 3.5 이하	33
3	BIO	3.5 초과	97
4	전체	학과 소계	196
5	CE	3.0 미만	66
6	CE	3.0 이상 3.5 이하	33
7	CE	3.5 초과	97
8	전체	학과 소계	196
9	CS	3.0 미만	65
10	CS	3.0 이상 3.5 이하	66
11	CS	3.5 초과	65
12	전체	학과 소계	196
13	EE	3.0 미만	66
14	EE	3.0 이상 3.5 이하	33
15	EE	3.5 초과	97
16	전체	학과 소계	196
17	HR	3.0 미만	10
18	HR	3.0 이상 3.5 이하	1
19	HR	3.5 초과	9
20	전체	학과 소계	20
21	ME	3.0 미만	66
22	ME	3.0 이상 3.5 이하	33
23	ME	3.5 초과	97
24	전체	학과 소계	196
25	전체	전체 총계	1,000

GROUP BY ROLLUP(major, gpa_tier) — GROUPING 값으로 상세·학과 소계·전체 총계를 구분하고 소계를 각 학과 아래에 배치했다.

WITH RECURSIVE · HIERARCHY

조직 계층 탐색과 직속 부하 집계

14

22 CEO부터 시작하는 조직 트리

```

--조직
emp 테이블의 조직 계층을 재귀적으로 조회한다.
데이터 구조:
- CEO는 manager_id = NULL이다.
- 매니저는 1명이다.
- 개발자는 30명이다.
- CEO - 매니저 - 개발자의 3단계 계층 구조이다.

요구사항:
- WITH RECURSIVE를 사용한다.
- CEO에서 시작하는 조직 트리를 탐색한다.
- 모든 직원의 계층 경로를 출력한다.
- 각 직원의 계층 경로를 출력한다.
- depth 컬럼을 포함한다.
- CEO의 depth는 0이다.
- path 컬럼을 포함한다.
- 경로 표현 매서는 다음과 같다.
- CEO > Mgr_2 > Dev_15
- 매니저의 직속 부하 직원 수를 집계하는 별도 쿼리도 작성한다.
- 직속 부하 직원 수의 집계명은 direct_reports로 한다.
*/

WITH RECURSIVE org_tree AS (
  SELECT
    emp_id,
    name,
    manager_id,
    0 AS depth,
    name::TEXT AS path
  FROM emp
  WHERE manager_id IS NULL

  UNION ALL

  SELECT
    e.emp_id,
    e.name,
    e.manager_id,
    ot.depth + 1,
    ot.path || ' > ' || e.name
  FROM emp e
  INNER JOIN org_tree ot ON e.manager_id = ot.emp_id
)
SELECT emp_id, name, depth, path
FROM org_tree
ORDER BY path;

SELECT
  m.emp_id AS manager_id,
  m.name AS manager_name,
  COUNT(e.emp_id) AS direct_reports
FROM emp m
LEFT JOIN emp e ON e.manager_id = m.emp_id
WHERE m.name LIKE 'Mgr_%' ESCAPE '\'
GROUP BY m.emp_id, m.name
ORDER BY m.emp_id;

```

WITH RECURSIVE — depth 0의 CEO부터 경로를 누적해 모든 직원의 계층을 출력했다.

22 매니저별 직속 부하 직원 수

```

--조직
emp 테이블의 조직 계층을 재귀적으로 조회한다.
데이터 구조:
- CEO는 manager_id = NULL이다.
- 매니저는 1명이다.
- 개발자는 30명이다.
- CEO - 매니저 - 개발자의 3단계 계층 구조이다.

요구사항:
- WITH RECURSIVE를 사용한다.
- CEO에서 시작하는 조직 트리를 탐색한다.
- 모든 직원의 계층 경로를 출력한다.
- 각 직원의 계층 경로를 출력한다.
- depth 컬럼을 포함한다.
- CEO의 depth는 0이다.
- path 컬럼을 포함한다.
- 경로 표현 매서는 다음과 같다.
- CEO > Mgr_2 > Dev_15
- 매니저의 직속 부하 직원 수를 집계하는 별도 쿼리도 작성한다.
- 직속 부하 직원 수의 집계명은 direct_reports로 한다.
*/

WITH RECURSIVE org_tree AS (
  SELECT
    emp_id,
    name,
    manager_id,
    0 AS depth,
    name::TEXT AS path
  FROM emp
  WHERE manager_id IS NULL

  UNION ALL

  SELECT
    e.emp_id,
    e.name,
    e.manager_id,
    ot.depth + 1,
    ot.path || ' > ' || e.name
  FROM emp e
  INNER JOIN org_tree ot ON e.manager_id = ot.emp_id
)
SELECT emp_id, name, depth, path
FROM org_tree
ORDER BY path;

SELECT
  m.emp_id AS manager_id,
  m.name AS manager_name,
  COUNT(e.emp_id) AS direct_reports
FROM emp m
LEFT JOIN emp e ON e.manager_id = m.emp_id
WHERE m.name LIKE 'Mgr_%' ESCAPE '\'
GROUP BY m.emp_id, m.name
ORDER BY m.emp_id;

```

direct_reports — 자기참조 LEFT JOIN으로 매니저별 직속 부하 수를 집계했다.

ROW_NUMBER · RANK · DENSE_RANK

학과별 GPA 상위 3명

15

23 서브쿼리 방식

```

-- 서브쿼리 방식
-- ROW_NUMBER()는 student_id로 동점 순서를 확정하고,
-- RANK()와 DENSE_RANK()는 GPA만 기준으로 계산해 동점 처리 차이를 비교한다.
SELECT
  student_id,
  name,
  major,
  gpa,
  row_num,
  rank_num,
  dense_rank_num,
  total_in_major
FROM (
  SELECT
    s.student_id,
    s.name,
    s.major,
    s.gpa,
    ROW_NUMBER() OVER (
      PARTITION BY s.major
      ORDER BY s.gpa DESC, s.student_id
    ) AS row_num,
    RANK() OVER (
      PARTITION BY s.major
      ORDER BY s.gpa DESC
    ) AS rank_num,
    DENSE_RANK() OVER (
      PARTITION BY s.major
      ORDER BY s.gpa DESC
    ) AS dense_rank_num,
    COUNT(*) OVER (
      PARTITION BY s.major
    ) AS total_in_major
  FROM student s
) ranked
WHERE row_num <= 3
ORDER BY major, row_num;

```

	student_id	name	major	gpa	row_num
1	29	Student_29	BIO	4.9	1
2	59	Student_59	BIO	4.9	2
3	89	Student_89	BIO	4.9	3
4	28	Student_28	CE	4.8	1
5	58	Student_58	CE	4.8	2
6	88	Student_88	CE	4.8	3
7	25	Student_25	CS	4.5	1
8	55	Student_55	CS	4.5	2
9	85	Student_85	CS	4.5	3
10	26	Student_26	EE	4.6	1
11	56	Student_56	EE	4.6	2
12	86	Student_86	EE	4.6	3
13	989	Student_989	HR	4.9	1
14	988	Student_988	HR	4.8	2
15	987	Student_987	HR	4.7	3
16	27	Student_27	ME	4.7	1
17	57	Student_57	ME	4.7	2
18	87	Student_87	ME	4.7	3

서브쿼리에서 학과별 순위와 전체 인원을 계산하고 외부 쿼리에서 상위 3명을 선택했다.

23 CTE 방식

```

-- CTE 방식
-- ROW_NUMBER()는 student_id로 동점 순서를 확정하고,
-- RANK()와 DENSE_RANK()는 GPA만 기준으로 계산해 동점 처리 차이를 비교한다.
WITH ranked_students AS (
  SELECT
    s.student_id,
    s.name,
    s.major,
    s.gpa,
    ROW_NUMBER() OVER (
      PARTITION BY s.major
      ORDER BY s.gpa DESC, s.student_id
    ) AS row_num,
    RANK() OVER (
      PARTITION BY s.major
      ORDER BY s.gpa DESC
    ) AS rank_num,
    DENSE_RANK() OVER (
      PARTITION BY s.major
      ORDER BY s.gpa DESC
    ) AS dense_rank_num,
    COUNT(*) OVER (
      PARTITION BY s.major
    ) AS total_in_major
  FROM student s
)
SELECT
  student_id,
  name,
  major,
  gpa,
  row_num,
  rank_num,
  dense_rank_num,
  total_in_major
FROM ranked_students
WHERE row_num <= 3
ORDER BY major, row_num;

```

	student_id	name	major	gpa	row_num
1	29	Student_29	BIO	4.9	1
2	59	Student_59	BIO	4.9	2
3	89	Student_89	BIO	4.9	3
4	28	Student_28	CE	4.8	1
5	58	Student_58	CE	4.8	2
6	88	Student_88	CE	4.8	3
7	25	Student_25	CS	4.5	1
8	55	Student_55	CS	4.5	2
9	85	Student_85	CS	4.5	3
10	26	Student_26	EE	4.6	1
11	56	Student_56	EE	4.6	2
12	86	Student_86	EE	4.6	3
13	989	Student_989	HR	4.9	1
14	988	Student_988	HR	4.8	2
15	987	Student_987	HR	4.7	3
16	27	Student_27	ME	4.7	1
17	57	Student_57	ME	4.7	2
18	87	Student_87	ME	4.7	3

동일 로직을 CTE로 분리해 단계별 의미와 재사용성을 명확하게 구성했다.

LAG · MAX · MIN

학생별 이전 과목 대비 성적 변화

16

24번 문자 성적을 숫자로 변환하고 이전 점수·차이·상태·점수 범위를 계산

Window Analysis

- 변환된 숫자 점수를 현재 점수로 사용한다.
- 학생별로 student_id를 기준으로 파티션을 나눈다.
- 과목 순서는 course를 기준으로 한다.
- LAG()를 이용해 이전 과목의 점수를 가져온다.
- 현재 점수와 이전 점수의 차이를 diff 컬럼으로 추가한다.
- 성적 변화 상태를 텍스트로 표시한다.
 - 상승
 - 유지
 - 하락
- 학생별 최고점과 최저점의 차이를 계산한다.
- 해당 컬럼명은 score_range로 한다.
- 최고점과 최저점 계산에는 Window Function을 사용한다.

```

*/
WITH scored AS (
    SELECT
        student_id,
        course,
        grade,
        CASE grade
            WHEN 'A' THEN 4
            WHEN 'B' THEN 3
            WHEN 'C' THEN 2
            WHEN 'D' THEN 1
        END AS current_score
    FROM enroll
),
grade_windows AS (
    SELECT
        student_id,
        course,
        grade,
        current_score,
        LAG(current_score) OVER (
            PARTITION BY student_id
            ORDER BY course
        ) AS previous_score,
        MAX(current_score) OVER (
            PARTITION BY student_id
        ) - MIN(current_score) OVER (
            PARTITION BY student_id
        ) AS score_range
    FROM scored
)
SELECT
    student_id,
    course,
    grade,
    current_score,
    previous_score,
    current_score - previous_score AS diff,
    CASE
        WHEN previous_score IS NULL THEN NULL
        WHEN current_score > previous_score THEN '상승'
        WHEN current_score = previous_score THEN '유지'
        ELSE '하락'
    END AS change_status,
    score_range
FROM grade_windows
ORDER BY student_id, course;

```

enroll 1 X

WITH scored AS (SELECT stu

Enter a SQL expression to filter results (use C ▶)

	123 student_id	AZ course	AZ grade	123 current_score	123 previous_
1	1	Course_3	C		2
2	1	Course_4	D		1
3	1	Course_5	A		4
4	2	Course_4	D		1
5	2	Course_5	A		4
6	3	Course_5	A		4
7	3	Course_6	B		3
8	3	Course_7	C		2
9	4	Course_6	B		3
10	4	Course_7	C		2
11	5	Course_7	C		2
12	5	Course_8	D		1
13	5	Course_9	A		4
14	6	Course_8	D		1
15	6	Course_9	A		4
16	7	Course_10	B		3
17	7	Course_11	C		2
18	7	Course_9	A		4
19	8	Course_10	B		3
20	8	Course_11	C		2
21	9	Course_11	C		2
22	9	Course_12	D		1
23	9	Course_13	A		4
24	11	Course_13	A		4
25	11	Course_14	B		3
26	11	Course_15	C		2
27	12	Course_14	B		3
28	12	Course_15	C		2
29	13	Course_15	C		2
30	13	Course_16	D		1
31	13	Course_17	A		4
32	14	Course_16	D		1
33	14	Course_17	A		4
34	15	Course_17	A		4
35	15	Course_18	B		3
36	15	Course_19	C		2
37	16	Course_18	B		3
38	16	Course_19	C		2
39	17	Course_1	A		4
40	17	Course_19	C		2

Refresh Save Cancel

200

200 rows fetched - 0.001s on 2026-07-23 at 15:53:55

LAG()로 이전 과목 점수를 가져오고, 학생별 MAX-MIN으로 score_range를 계산했다.

ROWS BETWEEN · CUMULATIVE · MOVING AVERAGE

주문 누적합과 이동평균

17

25 전체·고객별 누적합과 3개 주문 이동평균

```

SELECT
  order_id,
  customer_id,
  amount,
  SUM(amount) OVER (
    ORDER BY order_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumulative_amount,
  AVG(amount) OVER (
    ORDER BY order_id
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS moving_average_3,
  SUM(amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS customer_cumulative_amount
FROM orders
ORDER BY order_id
LIMIT 5;

```

	123 order_id	123 customer_id	123 amount	123 cumulative_amount	123 moving_average_3	123 customer_cumulative_a
1	1	2	18.01	18.01	18.01	18.01
2	2	3	31.02	49.03	24.515	31.02
3	3	4	44.03	93.06	31.02	44.03
4	4	5	57.04	150.1	44.03	57.04
5	5	6	70.05	220.15	57.04	70.05

ROWS BETWEEN으로 전체 누적합, 3개 행 이동평균, 고객별 누적 구매 금액을 함께 계산했다.

25 전체 합계 50% 초과 주문

```

WITH order_running AS (
  SELECT
    order_id,
    SUM(amount) OVER (
      ORDER BY order_id
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS cumulative_amount,
    SUM(amount) OVER () AS total_amount
  FROM orders
)
SELECT order_id
FROM order_running
WHERE cumulative_amount > total_amount * 0.5
ORDER BY order_id
LIMIT 1;

```

123 order_id	1,503
1	1,503

누적합과 전체 합계를 비교하고 order_id 오름차순의 첫 행만 선택했다.

FINAL REVIEW

최종 점검 및 실습 결론

18

1~4번 · JOIN 포함 범위

INNER, LEFT, FULL OUTER JOIN을 사용해 교집합, 기준 테이블 전체 및 고아 데이터를 구분했다.

5~13번 · 실무 JOIN과 집계

ANTI JOIN, DISTINCT, 고객 집계, SELF JOIN, 매핑 테이블 및 CROSS JOIN을 적용했다.

14~20번 · 서브쿼리와 집합

스칼라-상관 서브쿼리, EXISTS/NOT EXISTS 및 UNION으로 조건별 학생 집합을 구성했다.

21~25번 · 분석 SQL

ROLLUP, 재귀 CTE, 순위, LAG, 누적합과 이동평균 등 심화 기능을 구현했다.

요구사항 반영

문항별 필수 JOIN 유형, 집계 함수, 컬럼명, 정렬 기준과 출력 건수를 반영했다.

결과 컬럼 검토

학생·고객·직원 식별 정보와 문항이 요구한 계산 컬럼 중심으로 결과를 구성했다.

PostgreSQL 종합실습 2 완료

25개 문항의 SQL 작성과 실행 결과 확인을 완료했다. 관계형 데이터의 연결 범위부터 집계·계층·분석 쿼리까지 PostgreSQL의 핵심 기능을 실습 데이터에 적용했다.

INNER / OUTER JOIN

ANTI / SEMI JOIN

GROUP BY

SUBQUERY

CTE

WINDOW FUNCTION